



TECHQUEST

PROBLEM STATEMENT SERIES

PRODUCT DEVELOPMENT & AGENTIC AI · YELLOW.AI

Nexus Loop

Building an Autonomous Improvement Loop for AI Agent Deployments

FORMAT

Detect · Diagnose · Prescribe · Verify

TEAM STRUCTURE

Min. 1 2nd year

Min. 1 1st year

TEAM SIZE

7 per team

Introduction

A bank uses an AI agent to talk to its customers on WhatsApp. The agent answers questions, calls the bank's own APIs, reads a knowledge base, and hands over to a human when it gets stuck. Every one of those actions is logged. Millions of such conversations run every day, across hundreds of companies.

Yet when something breaks, a person finds out from a customer complaint. Then they guess what changed, read transcripts by hand, edit a prompt, and hope. Nothing checks whether the fix worked. The data to answer every one of those questions already exists. Nobody has built the thing that reads it. That is what you are building:

A system that reads an agent's logs, finds what broke, explains why, says who must act and what it costs if they don't, and, if you get that far, proposes a fix and proves it works.

Problem Statement Description

The Task in One Sentence is that from the given eight weeks of logs from an AI agent deployment you have never seen, produce:

1. *One report file* that says what broke, why, who must act, and what could not be measured at all.
2. *One screen* a person can read and act from.

TWO DELIVERABLES, TWO INSTINCTS

Engineering half

Product half



	Engineering half	Product half
What you deliver	<code>loop-report.json</code> , generated by your code, valid against the schema in the kit	One screen: a finding, the evidence behind it, and an approve/reject control for the proposed fix
The test	Did you find the real problems, avoid the fake ones, state your denominators honestly, and refuse what cannot be measured	Would an operator who was not on your team know what to do on Monday morning after reading it
Scored by	A script, 55 points . You have the exact script from day 1	People, 45 points , against the rubric on page 3

Both are required.

A report nobody can read is a script. A screen with nothing behind it is a mockup.

DATASET & PROBLEM CONTEXT

What Is Hidden in the Data

The data is generated, and problems have been deliberately planted in it. The practice data ships with the answer key so teams can check themselves throughout the week.

Planted	What it looks like	What we expect from you
3 real problems	A new product launches with nothing about it in the knowledge base. An API starts returning empty results while reporting success. A prompt change makes conversations twice as long without changing outcomes.	Find it. Name the slice of traffic it lives in. Name the cause and point at the config change behind it.
3 lookalikes	A marketing campaign shifts the mix of questions people ask. A flash sale spikes traffic and latency for four days. The quality-scoring rubric changes version halfway through.	Examine each and dismiss it in writing. Flagging one as a regression costs more than missing a real problem.
1 unanswerable question	"What is our failover rate?" The runtime has no failover mechanism, so no such event is ever logged.	Refuse it, and specify the exact event that would need to exist to answer it.
2 denominator traps	Legacy traffic logs no tool calls at all. One field is nearly unique per customer.	State the true coverage of any tool metric. Refuse the breakdown and cite the budget.

Each real problem is built to beat a lazy approach.



- One has no “before” period, so only a comparison against the deployment's own best conversations catches it.
- One is invisible in the obvious error-rate metric.
- One never moves the outcome metric at all.

LEVELS OF COMPLETION

How Far to Go: Three Levels

Work outward from the middle.

Level 1 is the target, and every team should reach it by day 3.

Level	What you have done	Machine score, of 55
Start	Ran starter.py. Two metrics, honest coverage, a valid file.	9.8
Level 1	Found one real problem, quantified it, explained it. Dismissed the three lookalikes. Refused the unanswerable question.	38.0
Level 2	Mined what "good" looks like for this deployment from its own best conversations. Proposed one fix and proved it on the replay service.	41.5
Level 3	Found the other problems too. Recorded a human decision on each fix. The system keeps a record of its own hit rate.	46.3

These are measured, not estimated.

Each level is a real example report in the kit, and teams can score themselves on day 1. Notice how small the gap is from Level 1 to Level 3. A Level 1 with an excellent screen and decision layer scores about 70 of 100. A Level 3 with neither scores about 58.

Depth on one problem beats breadth across all eight report sections.

A Finding Is a Decision, Not a Number

This is where most of the product points live. "Resolution in cohort X fell from 0.86 to 0.31" is true and useless. Every real problem you report must carry:

- **How many conversations** it affected, and what share of traffic that is.
- **What happened to those users.** How many would have resolved and did not. How many reached a human who was never meant to be involved.
- **How long** it has been running, and **what it has cost.**
- **Who has to act.** The agent builder, the business owner, or the platform team.
- **What happens if nobody acts.** One sentence, in plain words.



- **How you computed it.** A derivation someone can check against the same data, naming the baseline you chose and why. One problem has no before-period, so its baseline must be peer traffic. Another looks identical to its peers and is only visible against its own past.

Every figure is computed from the data. Nothing is asserted. If you propose a fix, frame it as a decision: what you are asking a human to approve, what the risk is if your diagnosis is wrong, and what would make you not ship it. Then put an **approve/reject** control on your screen and record what the human decided. A rejection with a good reason scores as well as an approval.

TWO RULES YOU CANNOT BREAK

Rule 1. Never ask a language model something the logs already record. Whether an API returned an error, which model served a turn, whether a handoff happened: these are facts in the data. Use a model only for questions of meaning: was the user satisfied, was the answer correct, what was this conversation about. Version any judge you build, and publish how often it agrees with the human labels in the kit. Breaking this rule is disqualifying.

Rule 2. Your system may propose changes to the agent, never to the yardstick. Metric definitions, judge versions, and the reviewed set of good conversations are fixed. A "fix" whose effect is that a number improved because its definition changed is disqualifying.

Because of these two rules, every number your system reports carries three honesty fields. **Fidelity:** measured from logs, judged by a model, or derived from other numbers. **Coverage:** what share of traffic can even produce this number. **Calibration,** for judged numbers: agreement with the human labels. The human labellers disagree with each other about 7.5% of the time, so a calibration of 1.00 is a bug, not a good judge.

Tasks

TASK 01 - Detection & Diagnosis

Analyze the eight weeks of AI agent deployment logs and identify genuine problems. Teams should:

- Find real problems.
- Identify the slice of traffic affected.
- Quantify the impact.
- Identify the cause.
- Point to the configuration change responsible where applicable.
- Examine the lookalikes.
- Explicitly dismiss false alarms.
- Identify what cannot be measured.

Deliverable



A valid loop-report.json containing the relevant findings, evidence, diagnoses, and measurement gaps.

TASK 02 - Standard & Fix Proposal

Mine what “good” looks like for the deployment from its own best conversations.
Use this standard to:

- Understand what the deployment should look like.
- Identify deviations from that standard.
- Propose a fix for a real problem.
- State the risk associated with the diagnosis.
- Define what would prevent the fix from being shipped.

Deliverable

A proposed fix represented in the report and presented through the screen for human approval or rejection.

TASK 03 - Fix Verification

Use the replay service to test a proposed fix.
The fix should be evaluated on the affected cohort using before-and-after results.
A wrong fix returning no effect is a valid result, not an error.

Deliverable

Evidence showing whether the proposed fix actually worked.

TASK 04 - Decision Layer & Self-Assessment

Complete the improvement loop by recording the human decision for each proposed fix.
At the highest level, the system should also keep a record of its own hit rate.

Deliverable

- Recorded human decisions.
- *Approve/reject* outcome.
- Self-assessment of the system's performance where applicable.

Report Structure

The report's eight sections represent the steps of the improvement loop.
Required Sections:



1. Metrics

What you measured. Every metric must include:

- Fidelity
- Coverage

2. Findings

What moved. This must include what you decided *was not a problem*.

3. Diagnoses

Why the problem happened. Include the configuration change you blame.

4. Gaps

What you could not measure. Include the exact event that would need to exist to answer the question.

Optional Sections

The remaining points live in:

1. Standard
2. Prescriptions
3. Verifications
4. Self Assessment

WHAT YOU GET

Everything lives in `tools/nexus-loop-kit/`. The kit uses *plain Python with no dependencies*. Run `make kit`. The practice data generates in approximately *20 seconds*.

PROVIDED FILES & TOOLS

File	What it is	How you use it
<code>kit/corpus/</code>	80,000 conversations, 880,000 execution steps, transcripts, 16 timestamped config changes, sparse ratings. Gzipped JSON lines.	Your input. Use <code>corpus_sample/</code> (a 5% slice) while iterating; confirm on the full set before reporting.
<code>kit/catalog.json</code>	What every field means, which joins are safe, what is measurable and what is not.	Read this before writing any query. Teams that skip it lose day 3 to joins that do not hold.
<code>kit/asks.md</code>	Eleven questions in an operator's own words. Four are required: A01, A04,	Your system answers them, or explains exactly why it cannot.



File	What it is	How you use it
	A09, A11.	
kit/labels/	400 human "was this resolved" labels and about 100 human quality scores.	Compute calibration for anything judged.
kit/ground_truth/	The answer key for the practice data.	Self-score all week. The day-6 data has its own key, which you never see.
starter.py	Reads the data, authors two honest metrics, writes a valid but nearly empty report.	Run it first, then read it. Its coverage handling is the pattern to copy.
score.py	The exact grading script.	Run it against your own report from day 2 onward. It is not a black box.
replay/serve.py	POST a proposed fix and a slice of traffic; it returns before and after. 40 attempts per team.	Prove fixes. A wrong fix returns no effect, and that is a result, not an error.
schema/loop-report.schema.json	Your deliverable's contract. Four sections required, four optional.	Validate against it.
examples/	Real reports at Level 1, 2 and 3.	Score them yourselves. Copy the shape of their impact and decision blocks.

DATASET

The corpus contains:

- 80,000 conversations
- 880,000 execution steps
- Transcripts
- 16 timestamped config changes
- Sparse ratings

The data is provided as *gzipped JSON lines*. Teams should use the *5% corpus_sample/* while iterating and confirm results on the full dataset before reporting.

REQUIRED OPERATOR QUESTIONS

kit/asks.md contains *eleven questions* in an operator's own words. Four are required:

- A01



- *A04*
- *A09*
- *A11*

The system must either:

1. Answer the question, or
2. Explain exactly why it cannot.

REPLAY SERVICE

The replay service is available at *replay/serve.py*. It accepts:

- A proposed fix
- A slice of traffic

It returns:

- Before
- After

Each team has *40 attempts*.

A wrong fix that returns no effect is a result, not an error.

SEALED RUN

Day 6: The Sealed Run

At *12:00 on day 6*, everything freezes.

Your system runs, untouched, on a second dataset with the same kinds of problems planted on different:

- Days
- Tenants
- Slices of traffic

Its answer key is held by the organisers.

That output is what gets scored.

A system that needs hand-holding to run scores as if it failed.

This rule is what rules out demo-ware without anyone having to police it.

Tasks



Points	Criterion	Scored by	What it asks
20	<i>Diagnostic Accuracy</i>	Script	Right problem, right slice of traffic, right cause, and how late you noticed
15	<i>Specificity</i>	Script	No false alarms on the lookalikes, with credit for dismissing them explicitly
12	<i>Honesty</i>	Script	Fidelity and coverage on every metric, a calibration that is not 1.00, the correct refusal with a usable event spec
8	<i>Loop Completeness</i>	Script	How much of detect, diagnose, prescribe and verify runs linked and unattended
18	<i>Decision Quality</i>	People	Could an operator act on each finding? Is every impact figure traceable to the data, with the baseline named?
15	<i>The Screen</i>	People	Can a stranger read a finding, see the evidence, and approve or reject the fix? Does every number carry its definition?
12	<i>Framing</i>	People	Did the demo open on the problem, show the refusal, state coverage out loud, and admit what was faked?

Total: 100 points

The scorer also prints a checklist of what is objectively present in the report:

- How many findings carry an impact block.
- How many show their working.
- How many fixes a human actually decided on.

Judges score the quality of those elements so that three judges reading the same work score it the same way.

YOUR FIRST FOUR HOURS

1. Run the Kit

Run *make kit*. Then *starter.py*. Then *score.py*. You now have a valid report and a printed summary of the data. You are on the board before writing a line.

2. Read the Kit

Read:



- catalog.json end to end
- asks.md
- starter.py

Do not code yet.

3. Take Ask A11 First

Work out from the catalog that it cannot be answered, and write the gap specification. It is the cheapest real scoring in the event, and it teaches Rule 1 better than any explanation.

4. Start with the Sample

On the *5% sample*, group resolution rate by:

- Intent
- Per tenant
- Per week

Something will stand out. Do not trust it yet. Three of the things that stand out are not problems.

THE SIX DAYS

Day	Focus	What it asks
<i>Day 1</i>	Read the catalog. Get one metric computing.	One number on screen, with fidelity and coverage attached
<i>Day 2</i>	Emit a valid report, however thin. Start mining the standard.	score.py returns a number. Any number.
<i>Day 3</i>	Detection and diagnosis. Meet the lookalikes.	A scorecard on the board: detections and false alarms, no narrative
<i>Day 4</i>	One fix, proposed and replayed.	Before and after on the affected cohort
<i>Day 5</i>	The screen and the approval gate.	The full loop runs on one problem with no human except the approval
<i>Day 6</i>	<i>12:00 freeze. Sealed run.</i>	<i>15:00 demos. Ten minutes to present, five minutes of questions</i>

SUBMISSION REQUIREMENTS

Teams must submit:

1. loop-report.json



The report generated by the system from the sealed run. It must be produced by the system *without edits*.

2. The System + Screen

The complete system, together with the screen. The screen must contain a working **APPROVE** / **REJECT** control. The human decision must be written back into the report.

3. One-Page Note

The note must explain:

- What is real.
- What is stubbed.
- What breaks at *100× the scale*.

Faking is fine in six days.
Hiding it is not.

DEMONSTRATION

Demo Format. *10 minutes*. Open on the problem from the *operator's side*, not on your architecture. The demonstration should show:

1. The finding and its impact.
2. The lookalike you correctly ignored.
3. The diagnosis.
4. The configuration change behind it.
5. The proposed fix.
6. Proof that the fix works.
7. The approval gate.
8. The one question your system refused to answer.
9. Why the system refused to answer it.

The refusal is the most memorable thirty seconds you have.

NOTE ON ASSUMPTIONS & USAGE

Teams must be transparent about what their system actually does. Clearly state:

- What is *real*.
- What is *stubbed*.
- What is *simulated*.
- What breaks at *100× the scale*.



The system may propose changes to the agent, but it must not change the yardstick. Metric definitions, judge versions, and the reviewed set of good conversations are fixed. Any system that changes the definition of a metric in order to make its result look better is disqualified.

CORE PRINCIPLES

The challenge is built around four stages:

Detect → *Diagnose* → *Prescribe* → *Verify*

The goal is not simply to find unusual numbers. The goal is to build a complete improvement loop in which the system:

- Finds real problems.
- Avoids false alarms.
- Quantifies impact honestly.
- Explains causes.
- Identifies who needs to act.
- Explains the consequences of inaction.
- Refuses questions that cannot be answered from the available data.
- Proposes fixes.
- Validates those fixes.
- Places the final decision in human hands.
- Records that decision.

One line version

An agent deployment records almost everything it does and learns nothing from it, because the improvement loop is closed by a human moving between six tools. Build the loop instead.